no i want to loop through each of the seasons

To **loop through each season dynamically** and find the **players with the greatest score increase** season-to-season, we need to structure the Cypher query as follows:

🔷 Query Goals

1. **Find players who played in multiple seasons.**
2. **Retrieve their scores for each season as separate columns.**
3. **Handle missing scores (`NULL`) for seasons they didn't play.**
4. **Calculate the increase in score season-to-season.**
5. **Sort by the largest increase.**

✅ Cypher Query: Looping Through Seasons

cypher Copy Edit

```cypher
WITH ["2021-2022", "2022-2023", "2023-2024", "2024-2025"] AS seasons

MATCH (p:Player)-[stats:PLAYS_IN]->(m)-[:IS_PART_OF]->(s:Season)
WHERE s.season IN seasons

// Compute average score for each player per season
WITH p, s.season AS season, AVG(stats.score) AS avg_score

// Convert multiple rows into a single row per player with seasons as columns
WITH p.name AS player,
     MAX(CASE WHEN season = "2021-2022" THEN avg_score ELSE NULL END) AS score_202
     MAX(CASE WHEN season = "2022-2023" THEN avg_score ELSE NULL END) AS score_202
     MAX(CASE WHEN season = "2023-2024" THEN avg_score ELSE NULL END) AS score_202
     MAX(CASE WHEN season = "2024-2025" THEN avg_score ELSE NULL END) AS score_202

// Compute score differences season to season
WITH player,
     score_2021_2022, score_2022_2023, score_2023_2024, score_2024_2025,
     score_2022_2023 - score_2021_2022 AS diff_21_22_23,
     score_2023_2024 - score_2022_2023 AS diff_22_23_24,
     score_2024_2025 - score_2023_2024 AS diff_23_24_25

// Find the greatest single season increase
WITH player,
     score_2021_2022, score_2022_2023, score_2023_2024, score_2024_2025,
     diff_21_22_23, diff_22_23_24, diff_23_24_25,
     GREATEST(diff_21_22_23, diff_22_23_24, diff_23_24_25) AS max_increase

ORDER BY max_increase DESC
LIMIT 10  // Get top 10 players with highest improvement

RETURN player,
       score_2021_2022, score_2022_20 score_2023_2024, score_2024_2025,
       diff_21_22_23, diff_22_23_24, diff_23_24_25,
```

Message ChatGPT

ChatGPT can make mistakes. Check important info.

```cypher
WITH p, s.season AS season, AVG(stats.score) AS avg_score

// Convert multiple rows into a single row per player with seasons as columns
WITH p.name AS player,
     MAX(CASE WHEN season = "2021-2022" THEN avg_score ELSE NULL END) AS score_202
     MAX(CASE WHEN season = "2022-2023" THEN avg_score ELSE NULL END) AS score_202
     MAX(CASE WHEN season = "2023-2024" THEN avg_score ELSE NULL END) AS score_202
     MAX(CASE WHEN season = "2024-2025" THEN avg_score ELSE NULL END) AS score_202

// Compute score differences season to season
WITH player,
     score_2021_2022, score_2022_2023, score_2023_2024, score_2024_2025,
     score_2022_2023 - score_2021_2022 AS diff_21_22_23,
     score_2023_2024 - score_2022_2023 AS diff_22_23_24,
     score_2024_2025 - score_2023_2024 AS diff_23_24_25

// Find the greatest single season increase
WITH player,
     score_2021_2022, score_2022_2023, score_2023_2024, score_2024_2025,
     diff_21_22_23, diff_22_23_24, diff_23_24_25,
     GREATEST(diff_21_22_23, diff_22_23_24, diff_23_24_25) AS max_increase

ORDER BY max_increase DESC
LIMIT 10  // Get top 10 players with highest improvement

RETURN player,
       score_2021_2022, score_2022_2023, score_2023_2024, score_2024_2025,
       diff_21_22_23, diff_22_23_24, diff_23_24_25,
       max_increase
```

🔍 How It Works

✅ **Dynamically loops through seasons**
✅ **Aggregates scores per player**
✅ **Computes season-to-season differences**
✅ **Finds the greatest increase per player**
✅ **Sorts by improvement and returns the top performers**

📌 Next Steps

- Run this in **Neo4j Browser** to validate.
- If you want **all players instead of the top 10**, remove `LIMIT 10`.

Let me know if you need adjustments! 🚀

Message ChatGPT

ChatGPT can make mistakes. Check important info.

ChatGPT used to help generate logic for how seasons are looped through in cypher query for greatest year on year defender growth

eError Traceback (most recent call last)
Cell In[283], line 3
1 # EXAMPLE USE: Visualisation of Shots on target in season 2024-2025
----> 3 Image("images/Starting XI Greatest target.png", width=700, height=350)

TypeError: 'module' object is not callable

The error `TypeError: 'module' object is not callable` occurs because `Image` is being treated as a module instead of an object. This is likely due to a naming conflict or incorrect import.

Fix 1: Ensure Proper Import of `IPython.display.Image`

You need to use `IPython.display.Image`, not `PIL.Image`. Try this:

```python
from IPython.display import Image

# Display image correctly
Image(filename="images/Starting XI Greatest target.png", width=700, height=350)
```

🔷 **Key Fix:** Ensure `Image` is imported from `IPython.display`, **not from PIL**.

Fix 2: Check for Variable Name Conflicts

If you've previously imported `Image` from `PIL`, it may have overridden the IPython `Image`. Check with:

```python
print(Image)
```

- If it prints something like `<module 'PIL.Image'>`, then `Image` is coming from `PIL`, which is incorrect for displaying images in Jupyter.
- Restart the kernel and re-run the correct `from IPython.display import Image` import.

Fix 3: If You're Using Matplotlib Instead

If you meant to display an image with **Matplotlib**, use:

```python
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
```

Message ChatGPT

ChatGPT can make mistakes. Check important info.

Recommended Fix

Try **removing duplicate imports and ensuring `Image` is correctly referenced**:

```python
import os
import requests
import pickle
import pandas as pd
import altair as alt
import matplotlib.pyplot as plt
import ipywidgets as widgets
from io import BytesIO
from PIL import Image as PILImage  # Rename to avoid conflict
from IPython.display import display, clear_output, Image  # Keep IPython's Imag
from ipywidgets import VBox, Button, Dropdown, Output
from matplotlib.offsetbox import OffsetImage, AnnotationBbox
from matplotlib.image import imread
from mplsoccer import VerticalPitch
from dotenv import load_dotenv
from neo4j import GraphDatabase
from neo4j.exceptions import ServiceUnavailable

# Load environment variables
load_dotenv(dotenv_path=os.path.join(os.getcwd(), 'config/.env'))

# Dropdown widgets
stat_options = [
    "goals", "assists", "penalties", "shots", "shots_on_target", "touches",
    "tackles", "interceptions", "blocks", "xg", "npxg", "xag", "sca", "gca",
    "passes_completed", "passes_attempted", "progressive_passes", "carries"
]
stat_dropdown = widgets.Dropdown(
    options=stat_options,
    value="goals",
    description="Stat:",
    style={'description_width': 'initial'}
)
season_dropdown = widgets.Dropdown(
    options=["2021-2022", "2022-2023", "2023-2024", "2024-2025"],
    value="2024-2025",
    description="Season:",
    style={'description_width': 'initial'}
)

# Ensure function `get_top_team_by_stat` exists before calling `widgets.interac
try:
    ui = widgets.interactive(get_top_team_by_stat, query_stat=stat_dropdown, se
    display(ui)
except NameError:
    print("Error: `get_top_team_by_stat` function is not defined.")
```

Message ChatGPT

ChatGPT can make mistakes. Check important info.

from mplsoccer import VerticalPitch
from utilities.cypher_query_loader import CypherQueryLoader
from dotenv import load_dotenv
load_dotenv(dotenv_path = os.path.join(os.getcwd(),'config/.env'))
and not when its these from matplotlib.offsetbox import
OffsetImage, AnnotationBbox
from matplotlib.image import imread
import requests
from io import BytesIO
from mplsoccer import VerticalPitch
import matplotlib.pyplot as plt
from PIL import Image
from neo4j import GraphDatabase
from mplsoccer import VerticalPitch
import matplotlib.pyplot as plt
import os
from dotenv import load_dotenv
load_dotenv(dotenv_path = os.path.join(os.getcwd(),'config/.env'))
import os
import ipywidgets as widgets
from IPython.display import display

Your issue likely stems from **namespace conflicts** and how Jupyter handles imports when executing `widgets.interactive()`. Let's break this down.

Potential Causes of the Issue

1. **Duplicate Imports (`load_dotenv` imported twice)**
   - `load_dotenv()` is imported twice in your first set of imports.
   - This shouldn't break the code but may cause unnecessary reloading.
2. **Conflicting `Image` Imports**
   - You import `Image` from both `PIL` and `IPython.display`:

     ```python
     from PIL import Image
     from IPython.display import Image
     ```

   - This means `Image` **from `PIL` may override the one from `IPython.display`**, causing issues when calling `Image()`.
3. **Conflicting `widgets` Imports**
   - You first import `ipywidgets as widgets`, then also:

     ```python
     from ipywidgets import VBox, Button, Dropdown, Output
     ```

   - While this is usually fine, it can sometimes cause conflicts in Jupyter Notebook's internal state.
4. **CypherQueryLoader (`from utilities.cypher_query_loader import CypherQueryLoader`)**

Message ChatGPT

ChatGPT can make mistakes. Check important info.

ChatGPT used to debug error caused by two librarys having "Image" object

```
avg_stat: avg_stat, img: img}}) AS midfielders

    return defenders,midfielders,forwards
    """
    with driver.session() as session:
        result = session.run(query)
        for record in result:
            defenders = record['defenders']
            midfielders = record['midfielders']
            forwards = record['forwards']

            visualise_team(defenders, midfielders,
forwards,query_stat)
    driver.close() why is excluded players not working?
```

Your **excluded players list** (`excluded_players`) isn't working properly likely because of **how Cypher handles collections across multiple** `WITH` **clauses**. Here's what might be going wrong:

---

1. Issue: `excluded_players` Doesn't Carry Over Properly

- In each `WITH` clause, `excluded_players` is updated, but it **isn't cumulative across all player types**.
- When transitioning between **defenders → forwards → midfielders**, the previous `excluded_players` list might not be correctly passed down.

2. Issue: `NOT fw.name IN excluded_players` Fails Because of Scope

- `excluded_players` only holds **the previous group's names**, not **all previously selected players**.
- Each `WITH` clause starts fresh, meaning `excluded_players` only contains the last set of players.

✅ Fix: Maintain a Persistent `excluded_players` List

Instead of **overwriting** `excluded_players` at each step, **accumulate** all selected players:

Fixed Cypher Query

cypher Copy Edit

```cypher
MATCH (def:Player)-[:IS_IN_SQUAD]->(squad:Squad)
WHERE TOUPPER(def.position) CONTAINS "B" AND squad.season = "{season}"
AND def.img IS NOT NULL
MATCH (def)-[stats:PLAYS_IN]-(m)-[:IS_PART_OF]-(s)
WHERE s.season = "{season}"
WITH def, AVG(stats.{query_stat}) AS avg_stat, def.img as img
ORDER BY avg_stat DESC
LIMIT 4
```

Message ChatGPT

ChatGPT can make mistakes. Check important info.

Gpt use to help debug glitch where player who play in multiple positions were not being excluded from appearing twice